

LGCC Encryption and Decryption Pseudocode:

LGCC_Encrypt(P, K):

Kbytes = UTF8_bytes(K)

m = len(Kbytes)

if m < 3:

 pad Kbytes with 00 until length 3

 m = 3

n = len(P)

prev = Kbytes[0] // initial chaining value for i = 0

for i from 0 to n-1:

 s = (Kbytes[i mod m] + i) mod 256

 g = Kbytes[(i+1) mod m]

 r = Kbytes[(i+2) mod m] AND 7

 t = (P[i] + s) mod 256 // Caesar base (byte shift)

 u = t XOR g // logic gate mixing

 v = u XOR prev // chaining (feedback)

 C[i] = ROTL8(v, r) // diffusion

 prev = C[i] // feedback update for next round

return C

LGCC_Decrypt(C, K):

Kbytes = UTF8_bytes(K)

m = len(Kbytes)

if m < 3:

 pad Kbytes with 00 until length 3

 m = 3

n = len(C)

prev = Kbytes[0] // same initial chaining value for i = 0

for i from 0 to n-1:

 s = (Kbytes[i mod m] + i) mod 256

 g = Kbytes[(i+1) mod m]

 r = Kbytes[(i+2) mod m] AND 7

 v = ROTR8(C[i], r) // undo rotation

 u = v XOR prev // undo chaining

 t = u XOR g // undo mixing

 P[i] = (t - s) mod 256 // undo Caesar shift

 prev = C[i] // feedback update for next round

return P